

Lab 6: Test Planning

Erik Pettersson(eri498) Ahmad Soltani(ahms698)

October 2025

Part 1

Code being tested

```
package com.thealgorithms.sorts;

/**
 * BubbleSort algorithm implemented using recursion
 */
public class BubbleSortRecursive implements SortAlgorithm {
    /**
     * @param array - an array should be sorted
     * @return sorted array
     */
    @Override
    public <T extends Comparable<T>> T[] sort(T[] array) {
        bubbleSort(array, array.length);
        return array;
    }

    /**
     * BubbleSort algorithm implements using recursion
     *
     * @param array array contains elements
     * @param len length of given array
     */
    private static <T extends Comparable<T>> void bubbleSort(T[] array, int len) {
        boolean swapped = false;
        for (int i = 0; i < len - 1; ++i) {
            if (SortUtils.greater(array[i], array[i + 1])) {
                SortUtils.swap(array, i, i + 1);
                swapped = true;
            }
        }
        if (swapped) {
            bubbleSort(array, len - 1);
        }
    }
}
```

Control Flow Graph

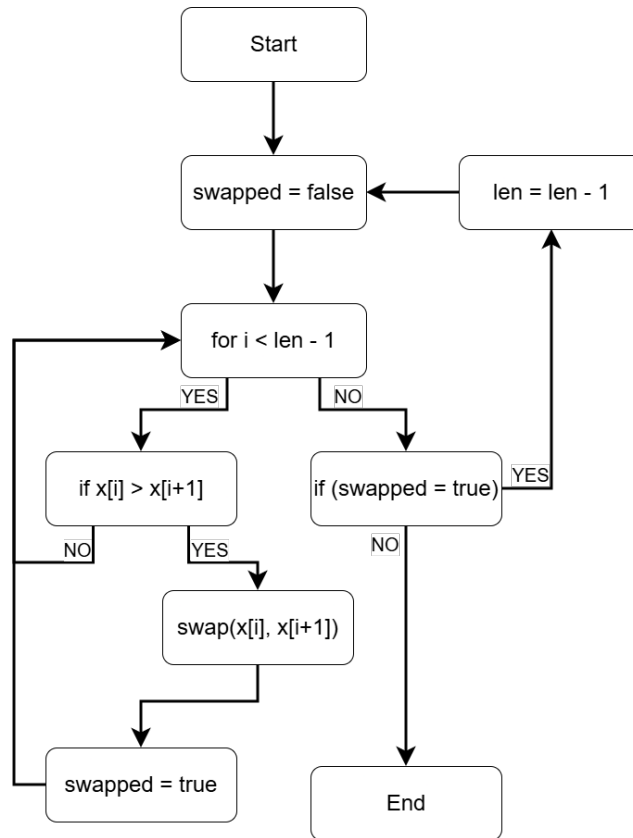


Figure 1: Control Flow Graph

Cyclomatic Complexity

Cyclomatic Complexity $M = E - N + 2P$

E = Number of edges

N = Number of nodes

P = Number of connected components

$M = 11 - 9 + 2 \times 1 = 4$

Basis paths

We need 4 independent paths:

Results from Test 4

Path ID	Description	Behavior
1	<code>Array.length() ≤ 1</code> .	Skips loop, swapped = false leads to no recursion.
2	Array already sorted.	Runs the loop, swapped = false leads to no recursion.
3	1 element out of order.	Runs loop, swapped = true gives 1 recursion.
4	Multiple elements out of order.	Runs loop, swapped = true, gives multiple recursions.

Test cases

Results from Test 4

Test ID	Input array	Expected Output	Basis Path	Explanation
1	[1]	[1]	1	$\text{len} \leq 1$, loop never runs.
2	[1, 2, 3]	[1, 2, 3]	2	Already sorted, no swaps made, not recursion triggered.
3	[3, 1, 2]	[1, 2, 3]	3	Loop runs and recursion is triggered. Array is sorted after first recursion so no more recursions will be triggered.
4	[3, 2, 1]	[1, 2, 3]	4	Loop runs and multiple recursions will trigger.

Part 2

Code being tested

```
package se.liu.ida.InsuranceApp.services;

import java.time.Year;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import se.liu.ida.InsuranceApp.data.ClientProfile;

@Service
public class InsuranceServiceImpl implements InsuranceService {

    @Autowired
    private ClientDataManagementService clientDB;

    @Override
    public int getClientDeductible(int clientId) {
        if (clientId < 0) {
            throw new IllegalArgumentException("clientId must be >= 0");
        }

        ClientProfile p = clientDB.findById(clientId);
        if (p == null) {
            throw new IllegalArgumentException("Client not found: " + clientId);
        }
        /**
         * assume getter exists
         */
        int now = Year.now().getValue();
        int age = now - p.getYearOfBirth();
        int yearsLicensed = now - p.getLicenseYear();
        int accidents = p.getAccidentsLastYear();
        boolean gold = p.isGoldMember();

        int deductible = 5000;

        if (age < 25 || yearsLicensed < 2) {
            deductible += 2000;
        }

        if (accidents >= 1) {
            int chargeable = Math.min(accidents, 2);
            deductible += 2000 * chargeable;
        }

        // Gold reduces deductible
        if (gold) {
            deductible -= 1000;
        }

        return deductible;
    }
}
```

Control Flow Graph

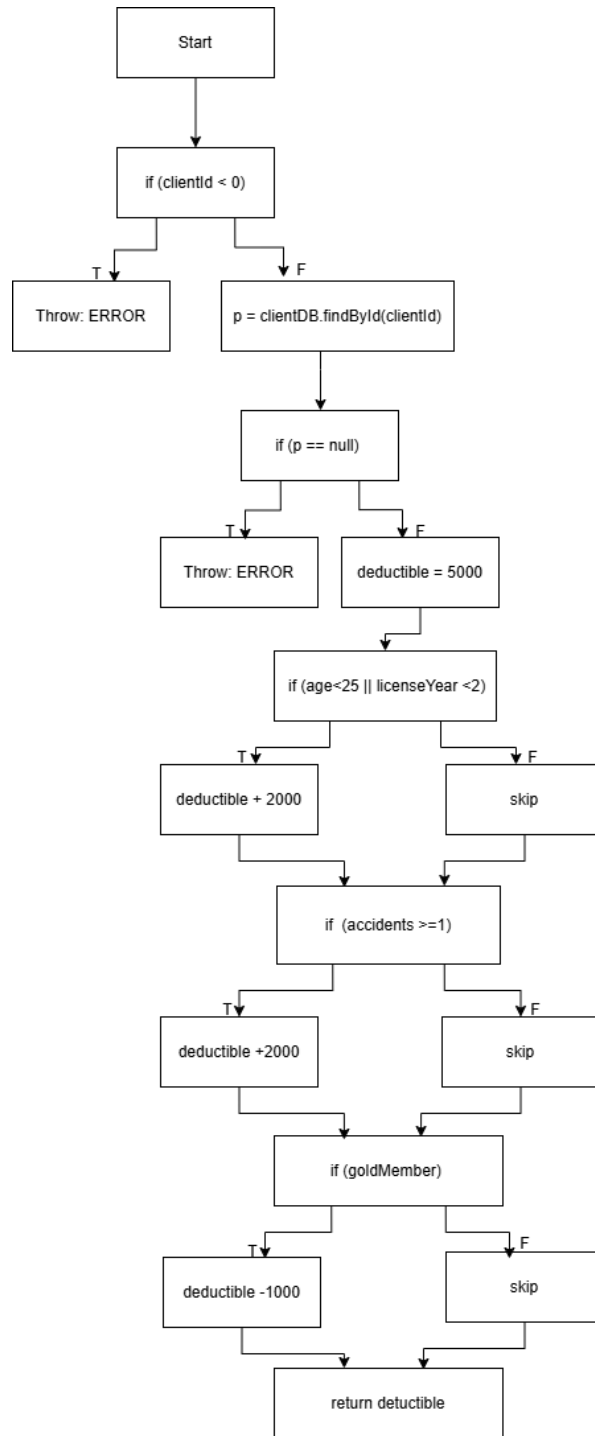


Figure 2: Control Flow Graph

Test case:

```
ClientProfile p = new ClientProfile();
    p.setYearOfBirth(2003);
    p.setLicenseYear(2024);
    p.setAccidents(2);
    p.setGoldMember(true);

when(clientDB.findById(42)).thenReturn(p);

int got = service.getClientDeductible(42);
assertEquals(10000, got); // 5000 + 2000 + 4000 - 1000 = 10000
```

Module Execution Paths (MEPs)

MEPs for getClientDeductible integration

MEP ID	Module	Module Path	Outcome	Condition / Notes
MEP(S,0)	Service (S)	$S \rightarrow S$	<i>throws</i>	$clientId < 0$ (precondition), no data call.
MEP(S,1)	Service (S)	$S \rightarrow D \rightarrow S$	call	Valid id; sends message <code>findById(clientId)</code> to D. (Call site is the sink MEP.)
MEP(S,2)	Service (S)	$S \rightarrow D \rightarrow S$	<i>throws</i>	Resume after return; $p == \text{null} \Rightarrow$ "Client not found".
MEP(S,3)	Service (S)	$S \rightarrow D \rightarrow S$	return	Resume; $p \neq \text{null}$. Compute (risk/accidents/gold) and return deductible.
MEP(D,1)	Data (D)	$D \rightarrow S$	return	<code>findById</code> : found $\Rightarrow$ returns <code>ClientProfile</code> .
MEP(D,2)	Data (D)	$D \rightarrow S$	return	<code>findById</code> : not found $\Rightarrow$ returns <code>null</code> .

MM-path graph

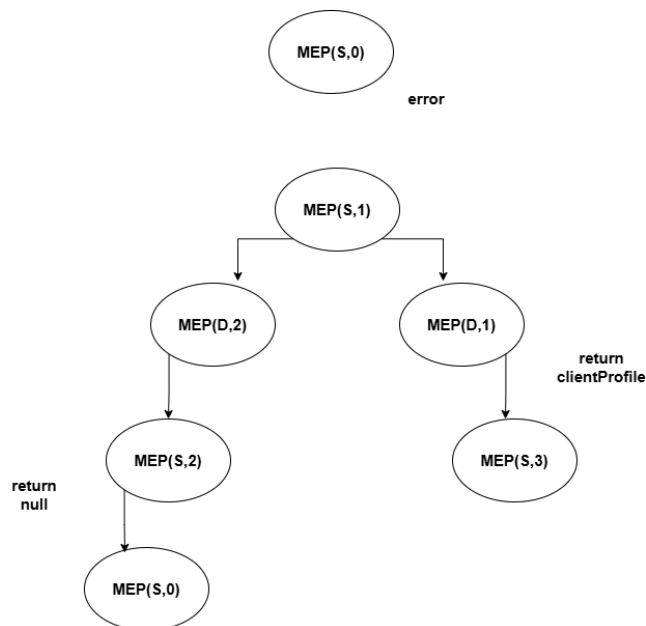


Figure 3: MM-path graph